

Spectral Decomposition

April 26, 2026

Python Code for Spectral Decomposition with Visualizations Spectral decomposition (also known as eigendecomposition) of a matrix involves decomposing a square matrix A into a set of eigenvalues and eigenvectors. The matrix A can be expressed as:

$$A = Q\Lambda Q^{-1}$$

where:

Λ is a diagonal matrix containing the eigenvalues

Q is a matrix whose columns are the eigenvectors of A

This code provides a Python implementation of spectral decomposition using NumPy and visualizes the results using Matplotlib.

0.1 Python Code: Spectral Decomposition with Visualizations

```
[1]: import numpy as np
import matplotlib.pyplot as plt

# Function to perform spectral decomposition
def spectral_decomposition(A):
    # Perform eigendecomposition
    eigenvalues, eigenvectors = np.linalg.eig(A)

    # Reconstruct the matrix from the eigenvalues and eigenvectors
    Lambda = np.diag(eigenvalues) # Diagonal matrix of eigenvalues
    Q = eigenvectors # Matrix of eigenvectors

    # Verify that A = Q * Lambda * Q^-1
    A_reconstructed = Q @ Lambda @ np.linalg.inv(Q)

    return eigenvalues, eigenvectors, A_reconstructed

# Example matrix
A = np.array([[4, 2],
              [1, 3]])

# Perform spectral decomposition
eigenvalues, eigenvectors, A_reconstructed = spectral_decomposition(A)
```

```

# Print results
print("Eigenvalues:")
print(eigenvalues)
print("\nEigenvectors:")
print(eigenvectors)
print("\nReconstructed Matrix (A):")
print(A_reconstructed)

# Visualize the matrix, eigenvalues, and eigenvectors
def visualize_spectral_decomposition(A, eigenvalues, eigenvectors):
    fig, ax = plt.subplots(1, 2, figsize=(12, 5))

    # Visualize the matrix A
    ax[0].imshow(A, cmap='coolwarm', interpolation='nearest')
    ax[0].set_title('Matrix A')
    ax[0].set_xticks(range(A.shape[0]))
    ax[0].set_yticks(range(A.shape[1]))

    # Visualize eigenvectors and eigenvalues
    ax[1].quiver(0, 0, eigenvectors[0, 0], eigenvectors[1, 0], angles='xy',
    ↪ scale_units='xy', scale=1, color='r', label=f"Eigenvalue: {eigenvalues[0]:.
    ↪ 2f}")
    ax[1].quiver(0, 0, eigenvectors[0, 1], eigenvectors[1, 1], angles='xy',
    ↪ scale_units='xy', scale=1, color='b', label=f"Eigenvalue: {eigenvalues[1]:.
    ↪ 2f}")
    ax[1].set_xlim([-1.5, 1.5])
    ax[1].set_ylim([-1.5, 1.5])
    ax[1].grid(True)
    ax[1].set_title('Eigenvectors and Eigenvalues')
    ax[1].legend()

    plt.show()

# Visualize the spectral decomposition results
visualize_spectral_decomposition(A, eigenvalues, eigenvectors)

```

Eigenvalues:

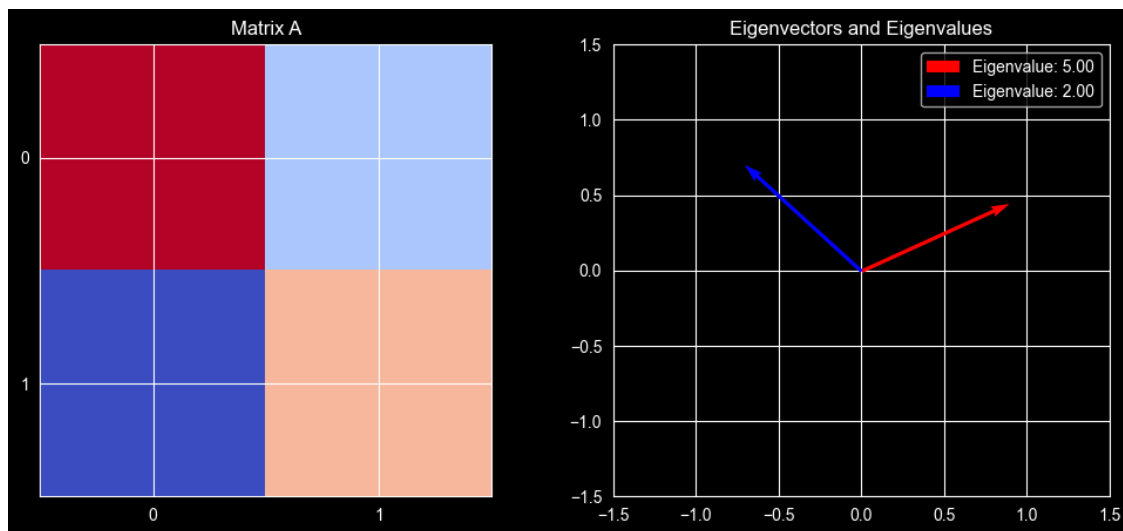
```
[5. 2.]
```

Eigenvectors:

```
[[ 0.89442719 -0.70710678]
 [ 0.4472136   0.70710678]]
```

Reconstructed Matrix (A):

```
[[4. 2.]
 [1. 3.]]
```



Explanation of the Code 1. Spectral Decomposition

The function `spectral_decomposition` takes a square matrix A and computes its eigenvalues and eigenvectors using `numpy.linalg.eig`.

It reconstructs the original matrix A using the formula

$$A = Q\Lambda Q^{-1}$$

and verifies if the decomposition is correct.

2. Visualization

Matrix Visualization: The matrix A is visualized using `imshow` to show a heatmap of its elements.

Eigenvalues and Eigenvectors Visualization: Eigenvectors are visualized as arrows in 2D space. The direction of the arrows represents the direction of the eigenvectors, and the length of each vector is proportional to the corresponding eigenvalue.

3. **Quiver Plot** The quiver function is used to draw the eigenvectors, and their length is scaled according to the eigenvalues.

[1]:

